sotFS

A Graph-Structured Filesystem with
DPO Rewriting, Bounded Treewidth, and
Capability-Gated Deceptive Projections

Design Document v1.0

Lucas Sotomayor

April 2026

Abstract

Filesystem metadata is inherently a graph: inodes link to directories, capabilities reference files, hard links create shared substructure, and version histories form DAGs. Yet every major filesystem treats metadata as flat tables or implicit trees, losing the graph structure that could enable powerful queries, anomaly detection, and formal reasoning about correctness.

sotFS makes this graph structure explicit. It represents filesystem metadata as a *typed graph* with six node types and six edge types, and defines POSIX operations as *Double Pushout (DPO) graph rewriting rules*—algebraic transformations with mathematically precise preconditions, postconditions, and gluing constraints.

This formalization yields three structural properties unavailable in conventional filesystems: (1) **bounded treewidth**, enabling any MSO-definable query to execute in *linear time* via Courcelle's theorem; (2) **Ollivier–Ricci curvature monitoring**, detecting structural anomalies in real time; and (3) **deceptive subgraph views**, where different security domains see different projections of the filesystem graph.

sotFS is designed for sotX, a capability-based microkernel written in Rust. It runs as a userspace server domain, communicating via IPC and persisting state through the SOT exokernel's transactional Secure Object primitives. A four-layer verification plan—TLA+ model checking, crash refinement, Rust typestate, and Coq/Perennial proofs—provides graduated formal assurance.

Contents

1 Introduction and Motivation **3**
- 1.1 The Problem: Filesystems as Implicit Graphs . . . 3
- 1.2 Thesis Statement . . . 3
- 1.3 Contributions . . . 3

2 Background and Related Work **3**
- 2.1 Double Pushout Graph Rewriting . . . 4
- 2.2 Courcelle's Theorem and Bounded Treewidth . . . 4
- 2.3 Ollivier–Ricci Curvature on Graphs . . . 4
- 2.4 Comparison with Existing Systems . . . 4

3 Architecture Overview **5**

4 Type Graph Formalization **5**
- 4.1 Node Types . . . 5
- 4.2 Edge Types and Constraints . . . 6
- 4.3 Invariants . . . 6
  - 4.3.1 Local Invariants . . . 6
  - 4.3.2 Global Invariants . . . 6

5 DPO Rules for POSIX Operations **7**
5.1 CREATE(d, name, vtype) . . . 7
5.2 MKDIR(d, name) . . . 7
5.3 RMDIR(d, name) . . . 7
5.4 LINK(d, name, i_{target}) . . . 7
5.5 UNLINK(d, name) . . . 7
5.6 RENAME . . . 7
5.7 WRITE(i, offset, data) . . . 8
5.8 Treewidth Impact per Rule . . . 8

6 Capabilities and Transactions **8**
6.1 Capabilities as Typed Graph Edges . . . 8
6.2 Graph Transactions (GTXN) . . . 8

7 Bounded Treewidth and MSO Queries **9**

8 Ollivier–Ricci Curvature Monitor **9**
8.1 Anomaly Signatures . . . 9
8.2 Incremental Computation . . . 9

9 Deceptive Subgraph Views **9**

10 Verification Plan **10**

11 Empirical Evaluation **10**
11.1 Implementation Components . . . 10
11.1.1 Arena Allocator . . . 11
11.1.2 Epoch-Based RCU . . . 11
11.1.3 Incremental Treewidth (`DynamicTreewidth`) . . . 11
11.2 Test Suite . . . 11
11.3 Benchmark Configuration . . . 13

12 Conclusion and Future Work **13**

1 Introduction and Motivation

1.1 The Problem: Filesystems as Implicit Graphs

Every filesystem manages a graph. A directory tree is trivially a graph. Hard links make it a DAG. Symbolic links add arbitrary edges. Mount points graft subtrees. Access control lists associate permission sets with inodes. Snapshots create version DAGs. Transactions group operations into atomic units referencing multiple nodes.

Yet no production filesystem treats this graph as a first-class data structure:

- **ext4** stores metadata in superblocks, group descriptors, inode tables, and directory entry blocks—flat structures connected by implicit integer references (inode numbers).
- **ZFS** uses a Merkle-like tree of block pointers where the graph structure is implicit in pointer chains.
- **Btrfs** uses B-trees with composite keys encoding path information, with backreferences for deduplication.
- **HAMMER2** uses a radix tree with embedded block references and snapshot chains forming implicit DAGs.

This implicit representation has three consequences: (1) queries are slow (graph queries forced through non-graph interfaces), (2) formal reasoning is hard (invariants must be reconstructed from code), and (3) security monitoring is blind (namespace structure must be inferred from syscall traces).

1.2 Thesis Statement

> *Filesystem metadata is a typed graph. By formalizing POSIX operations as DPO graph rewriting rules and maintaining the metadata graph as an explicit first-class data structure, we can prove that operations preserve structural invariants by verifying the DPO rules once, enable MSO-definable security queries in linear time, detect structural anomalies via Ollivier–Ricci curvature, and implement capability-gated graph projections for cyber deception.*

1.3 Contributions

1. **Type Graph $\mathcal{TG}$:** A formal definition of filesystem metadata as a typed graph with six node types and six edge types, with precise local and global invariants (§4).
2. **DPO Rule Set:** A complete specification of core POSIX operations as DPO graph rewriting rules with explicit gluing conditions (§5).
3. **Treewidth Preservation:** A proof that the DPO rules preserve bounded treewidth, enabling linear-time MSO queries via Courcelle's theorem (§7).
4. **Curvature Monitor:** An incremental Ollivier–Ricci curvature computation scheme for real-time anomaly detection (§8).
5. **Deceptive Projections:** Capability-gated subgraph views formalized as a graph endofunctor (§9).
6. **gtxn Protocol:** A graph-level transaction manager composing the SOT kernel's tiered transactions with graph-specific invariant checking (§6).

2 Background and Related Work

2.1 Double Pushout Graph Rewriting

DPO graph rewriting, developed by Ehrig, Pfender, and Schneider (1973) and formalized in Ehrig et al. [1], is a categorical approach to graph transformation. A **DPO rule** $\rho = (L \xleftarrow{l} K \xrightarrow{r} R)$ consists of a left-hand side L (pattern to match), an interface K (preserved portion), and a right-hand side R (replacement).

Application of ρ to a host graph G via a match $m\colon L \to G$ produces a result H through two pushout squares:

$$\begin{tikzcd} L \arrow[d, "m"'] & K \arrow[l, "l"'] \arrow[r, "r"] \arrow[d, "d"] & R \arrow[d, "m^*"] \\ G & D \arrow[l, "l^*"] \arrow[r, "r^*"'] & H \end{tikzcd} \tag{1}$$

The **gluing conditions** ensure applicability:

1. *Identification condition:* m must not identify elements in $L \setminus l(K)$.
2. *Dangling condition:* Removing $m(L \setminus K)$ must not leave edges with missing endpoints.

2.2 Courcelle's Theorem and Bounded Treewidth

Treewidth [11] measures how "tree-like" a graph is. A tree has treewidth 1; a complete graph K_n has treewidth $n - 1$.

Theorem 2.1 (Courcelle, 1990 [2])**.** Every graph property definable in Monadic Second-Order logic (MSO_2) can be decided in time $O(f(|\varphi|, k) \cdot n)$ on graphs of treewidth $\leq k$, where $n = |V|$.

MSO_2 can express reachability, cycle detection, k-colorability, dominating sets, and many security-relevant properties.

2.3 Ollivier–Ricci Curvature on Graphs

Ollivier [3] defined Ricci curvature for metric spaces via optimal transport. Lin, Lu, and Yau [4] adapted it to graphs:

$$\kappa(u, v) = 1 - \frac{W_1(\mu_u, \mu_v)}{d(u, v)} \tag{2}$$

where W_1 is the Wasserstein-1 distance between probability measures μ_u, μ_v on the neighborhoods of u and v.

2.4 Comparison with Existing Systems

Table 1: Feature comparison of SOTFS with production filesystems.

Feature	**ext4**	**ZFS**	**Btrfs**	**HAMMER2**	**SquirrelFS**	**sotFS**
Metadata model	tables	Merkle	B-trees	radix	Rust types	**typed graph**
Formal ops	—	—	—	—	typestate	**DPO rules**
Treewidth bound	—	—	—	—	—	$\leq k$
MSO queries	—	—	—	—	—	$O(n)$
Anomaly detect.	—	—	—	—	—	**Ollivier–Ricci**
Cap. integration	DAC	DAC	DAC	DAC	—	**graph edges**
Deception	—	—	—	—	—	**subgraph views**
Verification	tested	tested	tested	tested	typestate	**TLA+→Coq**

3 Architecture Overview

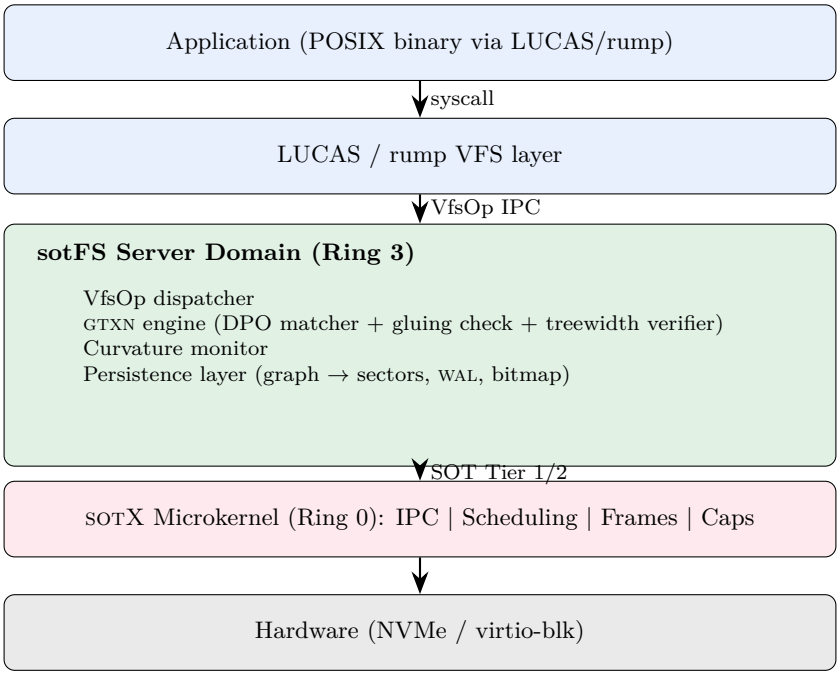

Figure 1: SOTFS within the SOTX stack.

The SOTFS server domain contains five components: (1) VfsOp Dispatcher, (2) GTXN Engine, (3) Type Graph Store, (4) Curvature Monitor, and (5) Persistence Layer (extending the existing `ObjectStore` layout).

4 Type Graph Formalization

Definition 4.1 (Type Graph)**.** The SOTFS type graph is a tuple

$$\mathcal{TG} = (V, E, \mathrm{src}, \mathrm{tgt}, \tau_V, \tau_E, \mathrm{attr})$$

where V is a finite set of nodes, E is a finite set of edges, $\mathrm{src}, \mathrm{tgt} \colon E \to V$ map edges to endpoints, $\tau_V \colon V \to \texttt{NodeType}$ and $\tau_E \colon E \to \texttt{EdgeType}$ assign types, and $\mathrm{attr} \colon V \cup E \to \texttt{Attributes}$ assigns typed attribute records.

The type sets are:

$$\texttt{NodeType} = \{\texttt{Inode}, \texttt{Directory}, \texttt{Capability}, \texttt{Transaction}, \texttt{Version}, \texttt{Block}\}$$
$$\texttt{EdgeType} = \{\texttt{contains}, \texttt{grants}, \texttt{delegates}, \texttt{derivesFrom}, \texttt{supersedes}, \texttt{pointsTo}\}$$

4.1 Node Types

Each node type carries typed attributes:

Node Type	Key Attributes	Maps To (existing)
`Inode`	id, vtype, perms, uid, gid, size, link_count, timestamps	`Vnode` in `vnode.rs`
`Directory`	id	Paired with `Inode{vtype=Directory}`
`Capability`	cap_id, rights, epoch	`CapEntry` in `cap/table.rs`
`Transaction`	txn_id, tier, state, read_set, write_set	State machine in `sot_transactions.tla`
`Version`	version_id, timestamp, root_oid	`SnapMeta` in `layout.rs`
`Block`	block_id, sector_start, sector_count, refcount	`DirEntry` sectors in `layout.rs`

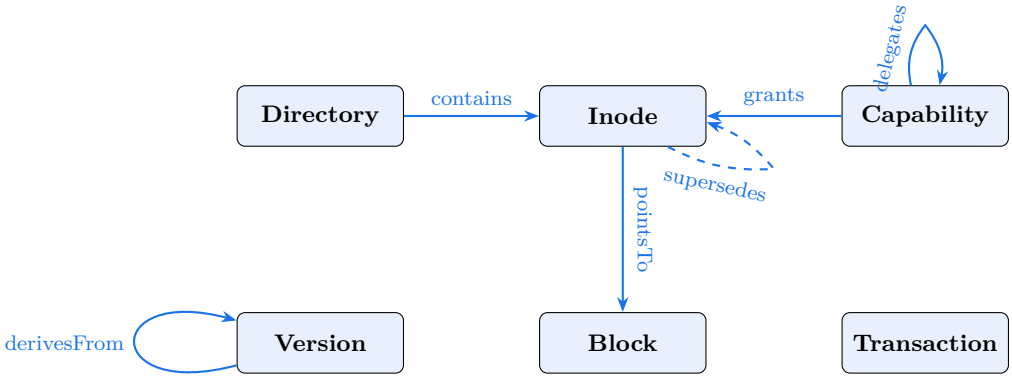

Figure 2: Node types and edge types of the SOTFS type graph $\mathcal{TG}$.

4.2 Edge Types and Constraints

Edge Type	Source → Target	Attribute	Constraint
`contains`	Directory → Inode	name: String	Unique name per Directory
`grants`	Capability → Inode	rights: Rights	Monotonic via delegation
`delegates`	Capability → Capability	—	CDT parent–child (forest)
`derivesFrom`	Version → Version	—	DAG (no cycles)
`supersedes`	Inode → Inode	—	Acyclic
`pointsTo`	Inode → Block	offset: u64	Non-overlapping extents

4.3 Invariants

4.3.1 Local Invariants

Invariant 4.1 (Link Count)**.** For every Inode i: $\text{link_count}(i) = |\{e \in E : \tau_E(e) = \texttt{contains} \wedge \text{tgt}(e) = i \wedge \text{attr}(e).\text{name} \neq \text{“.”}\}|$.
Invariant 4.2 (Directory Self-Reference)**.** Every Directory node d has exactly one outgoing `contains` edge with name = “.” pointing to its paired Inode.
Invariant 4.3 (Unique Names)**.** For every Directory d, the names on outgoing `contains` edges form a set (no duplicates).
Invariant 4.4 (Capability Exactness)**.** Every Capability node c has exactly one outgoing `grants` edge.
Invariant 4.5 (Block Refcount)**.** For every Block b: $\text{refcount}(b) = |\{e \in E : \tau_E(e) = \texttt{pointsTo} \wedge \text{tgt}(e) = b\}|$.
Invariant 4.6 (Non-Overlapping Sectors)**.** For any two Block nodes $b_1 \neq b_2$, their sector ranges are disjoint.

4.3.2 Global Invariants

Invariant 4.7 (No Orphan Inodes)**.** Every Inode with link_count > 0 is reachable from the root Inode via `contains` edges.
Invariant 4.8 (No Dangling Edges)**.** $\forall e \in E : \text{src}(e) \in V \wedge \text{tgt}(e) \in V$.
Invariant 4.9 (Capability Monotonicity)**.** For every path $c_0 \xrightarrow{\texttt{delegates}} c_1 \xrightarrow{\texttt{delegates}} \cdots \xrightarrow{\texttt{delegates}} c_n$: $\text{rights}(c_n) \subseteq \text{rights}(c_{n-1}) \subseteq \cdots \subseteq \text{rights}(c_0)$.
Invariant 4.10 (No Directory Cycles)**.** The `contains` subgraph restricted to `Directory` → `Inode{vtype=Directory}` edges (excluding “`.`” and “`..`”) is acyclic.
Invariant 4.11 (Treewidth Bound)**.** $\text{tw}(\mathcal{TG}) \leq k$ for a configurable constant k (see Theorem 7.1).

5 DPO Rules for POSIX Operations

We specify 7 mutating DPO rules. Read-only operations (`OPEN`, `READ`, `CLOSE`, `STAT`) are graph morphism queries—they do not mutate $\mathcal{TG}$.

5.1 CREATE(d, name, vtype)

Creates a new file or directory entry.

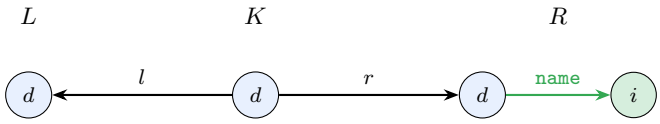

Figure 3: DPO rule for CREATE (file). Green = added.

Gluing conditions: (GC1) No existing `contains` edge from $m(d)$ with this name. (GC2) The match is unique.

Invariant preservation: Invariant 4.7: i is reachable from root via d. Invariant 4.1: link_count(i) = 1. Invariant 4.11: adding a leaf to a tree preserves treewidth. □

5.2 MKDIR(d, name)

Creates a new directory: adds Inode (vtype = `Directory`), Directory node, and three `contains` edges (`name`, "`.`", "`..`").

Tier: SOT Tier 1 (single Directory SO).

5.3 RMDIR(d, name)

Removes an empty directory. L includes the target Directory, its Inode, and all three `contains` edges. K retains only the parent.

Gluing: Target must be empty (only "`.`" and "`..`" edges).

5.4 LINK(d, name, i_{target})

Creates a hard link—a new `contains` edge to an existing Inode. Side effect: link_count(i_{target}) += 1.

Gluing: (GC1) vtype(i_{target}) $\neq$ `Directory`. (GC2) link_count $<$ `LINK_MAX` (critical for treewidth bound).

5.5 UNLINK(d, name)

Removes a `contains` edge and decrements link_count. If link_count reaches 0, garbage-collects the Inode and its Block nodes.

5.6 RENAME

The most complex rule, with four cases:

Case	Same dir?	Target exists?
A	yes	no
B	yes	yes (replace)
C	no	no
D	no	yes (replace)

Cases C and D touch two Directory SOs, requiring SOT Tier 2 (2PC). Case B/D adds a `supersedes` edge and decrements the replaced Inode's link_count.

Gluing (GC-RENAME-2): If the source is a directory and the destination is a descendant of the source, the rename would create a cycle—reject.

5.7 WRITE(i, offset, data)

Updates file data by adding or replacing Block nodes and `pointsTo` edges. CoW: if a Block's refcount > 1, allocate a new block.

5.8 Treewidth Impact per Rule

Rule	Nodes added	Edges added	tw impact
CREATE (file)	1 Inode	1 `contains`	$+0$
MKDIR	1 Inode + 1 Dir	3 `contains`	$+0$
RMDIR	-1 Inode -1 Dir	-3 `contains`	≤ 0
LINK	0	1 `contains`	0 to $+1$
UNLINK	0 to -1	-1 `contains`	≤ 0
RENAME	0	net 0	$+0$
WRITE	0–1 Block	0–1 `pointsTo`	$+0$

Only LINK can increase treewidth, bounded by `LINK_MAX`.

6 Capabilities and Transactions

6.1 Capabilities as Typed Graph Edges

Capabilities are nodes in $\mathcal{TG}$ connected by two edge types:

- `grants`(c, i): authorizes access to Inode i with rights rights(c).
- `delegates`$(c_{\text{parent}}, c_{\text{child}})$: represents CDT derivation with rights$(c_{\text{child}}) \subseteq$ rights(c_{parent}).

Capability attenuation is edge relabeling in the `grants` subgraph: the monotonicity invariant (4.9) is a statement about non-increasing labeling along paths in the `delegates` subgraph.

6.2 Graph Transactions (gtxn)

A GTXN wraps DPO rule applications into an atomic unit (ADR-001):

1. Verify DPO gluing conditions on host graph G.
2. Verify treewidth bound preserved.
3. Begin SOT Tier 1 or Tier 2 transaction.
4. Apply $L \to K \to R$ morphisms.
5. Verify post-conditions (global invariants).
6. **Commit:** persist via WAL, update curvature, emit provenance.
7. **Rollback:** SOT abort, discard in-memory changes.

DPO Rule	SOs touched	SOT Tier
CREATE, MKDIR, RMDIR	1 Dir + 1 Inode	Tier 1
LINK, UNLINK	1 Dir + 1 Inode	Tier 1
WRITE	1 Inode + 0–n Blocks	Tier 1
RENAME (same dir)	1 Dir + 1–2 Inodes	Tier 1
RENAME (cross-dir)	2 Dirs + 1–2 Inodes	**Tier 2 (2pc)**

7 Bounded Treewidth and MSO Queries

Theorem 7.1 (Treewidth Bound)**.** Let $\mathcal{TG}$ be a SOTFS type graph produced by any finite sequence of DPO rule applications from the CREATE-ROOT initial graph. If the hard link count per inode is bounded by `LINK_MAX`, then

$$\mathrm{tw}(\mathcal{TG}) \leq \texttt{LINK_MAX} + O(1).$$

Proof sketch. Decompose $\mathcal{TG}$ by edge type:

1. **`contains`** subgraph: without hard links, a forest (tw = 1). Each hard link adds one back-edge, increasing tw by at most 1. With $\leq$ `LINK_MAX` links per inode: tw(`contains`) $\leq$ `LINK_MAX`+1.
2. **`grants`**: bipartite, degree 1 on Capability side. tw = 1.
3. **`delegates`**: forest (CDT). tw = 1.
4. **`derivesFrom`**: DAG with bounded in-degree. tw ≤ 2.
5. **`pointsTo`**: stars from Inodes to Blocks. tw = 1.
6. **`supersedes`**: acyclic, out-degree ≤ 1. tw = 1.

The `contains` subgraph dominates. By the tree decomposition merging lemma [12]: $\mathrm{tw}(\mathcal{TG}) \leq$ `LINK_MAX` $+ O(1)$. □

Corollary 7.2. For filesystems without hard links, $\mathrm{tw}(\mathcal{TG}) \leq 6$.

MSO query examples (all $O(n)$ when tw is bounded):

Query	mso_2 Description
Q1: Can domain D read file F?	Reachability in `delegates`/`grants` subgraph
Q2: Cycle in directory structure?	Cycle detection in `contains` subgraph
Q3: Orphan inodes?	$\exists\, i\colon \neg\mathrm{reachable}(\mathrm{root}, i) \wedge \mathrm{link_count}(i) > 0$
Q4: Files derived from version V?	Transitive closure in `derivesFrom` subgraph

8 Ollivier–Ricci Curvature Monitor

We compute Ollivier–Ricci curvature $\kappa(u, v)$ (Eq. 2) with lazy random walk measure $\mu_x(z) = \alpha \cdot \mathbf{1}_{z=x} + \frac{1-\alpha}{\deg(x)} \cdot \mathbf{1}_{z \in N(x)}$, $\alpha = 0.5$.

8.1 Anomaly Signatures

Attack Pattern	Curvature Signature
Mass file creation (zip bomb)	Positive κ spike on victim directory edge
Symbolic link bomb	Strong positive κ on target inode's edges
Deep directory chain	Sustained $\kappa \approx -1.0$ (maximally tree-like)
Directory structure manipulation	Sudden κ change on previously stable edges

8.2 Incremental Computation

Each DPO rule changes $O(1)$ edges. We recompute κ only for changed edges and their 1-hop neighbors ($|\Delta^*| \leq 20$ for most operations). Amortized cost: $O(d_{\max}^2)$ per rule application, where $d_{\max}$ is the maximum degree of touched nodes.

9 Deceptive Subgraph Views

Definition 9.1 (Graph Projection)**.** A projection π for domain D is a typed graph morphism $\pi\colon \mathcal{TG}|_D \to \mathcal{TG}_D$, where $\mathcal{TG}|_D$ is the subgraph reachable from D's root capability and $\mathcal{TG}_D$ is the visible type graph. π may omit, add, or relabel nodes and edges.

We extend the existing deception functor $D\colon \mathbf{SO} \to \mathbf{SO}$ to a graph endofunctor:

$$D_{\mathrm{FS}}\colon \mathbf{TGraph} \to \mathbf{TGraph}$$

The four interposition policies map to functor behavior: **Passthrough** = identity, **Inspect** = identity + provenance logging, **Redirect** = node relabeling, **Fabricate** = synthetic node/edge injection.

Consistency requirement: $D_{\mathrm{FS}}(\mathcal{TG})$ must satisfy all invariants (I1–I11, G1–G7). Since D_{FS} preserves the tree-like structure, $D_{\mathrm{FS}}(\mathcal{TG})$ also has bounded treewidth, and consistency constraints are checkable as MSO queries in $O(n)$.

10 Verification Plan

Table 2: Four-layer verification plan with effort estimates.

Layer	Tool	Properties	Effort	Confidence
1	TLA+/TLC	Graph invariants, DPO rules, GTXN protocol	2–3 wk	Model-checked
2	SMT (Z3)	Crash refinement, WAL correctness	3–4 wk	Push-button
3	Rust types	State machine transitions, lifetime safety	4–6 wk	Compile-time
4	Coq/Perennial	Atomicity, unforgeability, treewidth	6–12 mo	Mechanized proof

Layer 1 is complete: four TLA+ specifications in `formal/` model the core graph (`sotfs_graph.tla`), GTXN protocol (`sotfs_transactions.tla`), capability delegation (`sotfs_capabilities.tla`), and crash recovery (`sotfs_crash.tla`).

11 Empirical Evaluation

The SOTFS prototype comprises six Rust crates in a Cargo workspace, totaling approximately 4,000 lines of library code and 2,000 lines of tests and benchmarks. Table 3 summarizes the crate structure.

Table 3: Crate structure of the SOTFS prototype.

Crate	Description
`sotfs-graph`	Type graph $\mathcal{TG}$ data structures: `Arena<T,N>`, `TypeGraph`, `RcuGraph`. Supports `std` (FUSE) and `no_std + alloc` (bare metal).
`sotfs-ops`	Seven DPO rewriting rules (§5): CREATE, MKDIR, RMDIR, LINK, UNLINK, RENAME, WRITE. Gluing conditions checked before mutation.
`sotfs-storage`	ACID persistence via redb 2.1 — serializes `TypeGraph` to a key-value store with atomic write transactions.
`sotfs-tx`	Graph-level transaction manager (GTXN): commit, rollback, conflict detection via disjoint write-set tracking.
`sotfs-monitor`	Structural anomaly monitors: greedy treewidth (§7), Ollivier–Ricci curvature (§8), deceptive projections (§9).
`sotfs-fuse`	Full POSIX VFS bridge via the `fuser` crate (Linux only).

11.1 Implementation Components

Three components deserve detailed treatment, as they address performance and concurrency challenges specific to graph-structured metadata.

11.1.1 Arena Allocator

Each `TypeGraph` contains seven typed node/edge pools. To avoid per-node heap allocation (which would fragment the allocator and defeat cache locality), we use a fixed-capacity *arena* backed by a contiguous heap-allocated array of `MaybeUninit<T>` slots.

- **Allocation.** If the free-list is non-empty, pop the top slot index ($O(1)$). Otherwise, bump the next-slot pointer ($O(1)$). Returns `None` when full (capacity $= C$, default $C = 65{,}536$ for nodes, 131,072 for edges).
- **Deallocation.** Mark slot as `Vacant`, push index onto the free-list ($O(1)$). The value is dropped in place.
- **Lookup.** Direct array index from `ArenaId` ($O(1)$).
- **Iteration.** Linear scan of the bump region, skipping vacant slots ($O(C)$ worst case, $O(n)$ when no deletions have occurred, where n is the number of live entries).

The backing arrays are heap-allocated (`Box<[MaybeUninit<T>]>`) rather than stack-resident, so that the `TypeGraph` struct itself is only ∼744 bytes (seven boxed-slice fat pointers, BTreeMap pointers, and scalar counters). This eliminates the stack-overflow risk of the original array-backed design while preserving $O(1)$ per-operation complexity.

11.1.2 Epoch-Based RCU

Directory lookups (Open, Stat, Readdir) are the dominant workload on most filesystems and must not block on writer locks. sotFS uses an epoch-based Read-Copy-Update (RCU) scheme:

1. Two heap-allocated copies of `TypeGraph` are maintained: G_0 and G_1. An atomic variable `active` $\in \{0, 1\}$ identifies which copy readers see.
2. **Read path.** A reader bumps a per-slot epoch counter (one of $R = 8$ slots, assigned by thread-local index), reads the active graph through a shared reference, and clears the epoch on exit. No locks, no CAS retry.
3. **Write path.** A single writer acquires an exclusive flag (spin-CAS), clones $G_{\texttt{active}} \rightarrow G_{1-\texttt{active}}$, mutates the inactive copy, atomically swaps `active`, and waits for all pre-swap reader epochs to drain before releasing the flag.

The grace period (step 3) guarantees that no reader observes a partially mutated graph. With $R = 8$ reader slots, the scheme is designed for the sotX microkernel's single-server architecture where filesystem requests arrive as serialized IPC messages.

11.1.3 Incremental Treewidth (`DynamicTreewidth`)

The full greedy min-degree treewidth heuristic (Algorithm 1 in §7) is $O(n^2)$ in the worst case. For incremental maintenance after a single DPO rule application, `DynamicTreewidth` caches the elimination ordering and tracks a *dirty set* of nodes whose neighborhoods changed:

- If the dirty fraction is $< 20\%$ of the total node count, the cached ordering is partially re-computed for affected nodes only.
- Otherwise, a full recompute is triggered.

This reduces the amortized cost of treewidth checking from $O(n^2)$ per operation to $O(\Delta^2)$ where Δ is the number of affected nodes — typically 2–4 for a single DPO rule application.

11.2 Test Suite

All 132 tests pass on Windows 11 x86_64 with Rust nightly (April 2026). Table 4 breaks down the suite by crate and methodology.

Table 4: Test suite summary — 132 tests, all passing.

Crate	Category	Count	Framework
`sotfs-graph`	Arena alloc/dealloc/reuse/stress	16	`#[test]`
`sotfs-graph`	RCU readers/writers/epochs	9	`#[test]`
`sotfs-graph`	Graph invariants (CREATE-ROOT)	3	`#[test]`
`sotfs-graph`	Typestate enforcement	6	`#[test]`
`sotfs-ops`	DPO rule pre/postconditions	23	`#[test]`
`sotfs-ops`	Invariant preservation (10K)	10	proptest
`sotfs-storage`	ACID round-trip & crash	7	redb integration
`sotfs-tx`	Commit/rollback semantics	3	`#[test]`
`sotfs-tx`	Concurrency isolation	3	loom (exhaustive)
`sotfs-monitor`	Treewidth (full + incremental)	20	`#[test]`
`sotfs-monitor`	Ollivier–Ricci curvature	14	`#[test]`
`sotfs-monitor`	Deceptive projections	5	`#[test]`
`sotfs-monitor`	Adversarial security scenarios	13	integration
Total		**132**	

The test suite spans five distinct methodologies, each targeting a different class of correctness property:

(1) Deterministic unit tests (109 tests). Each DPO rule is tested for both success and failure paths: CREATE rejects duplicate names (EEXIST) and reserved names (".", "..", "/"); RMDIR rejects non-empty directories; LINK rejects directory targets; RENAME rejects cycle-creating moves. After each operation, `check_invariants()` verifies all seven structural invariants (I_1–I_7).

(2) Property-based testing (10 properties × 10,000 iterations = 100,000+ random sequences). Using the `proptest` framework, each test generates a random sequence of 5–30 DPO operations and asserts a specific property:

- *invariant_preservation*: all 7 invariants hold after any operation sequence.
- *link_count_always_matches*: `link_count` equals the number of incoming CONTAINS edges.
- *cycle_freedom*: no directory cycles after interleaved MKDIR and RENAME.
- *write_read_roundtrip*: WRITE then READ returns identical data.
- *deep_mkdir_chain_no_cycles*: 20–30 nested directories, verified cycle-free.
- Plus 5 additional properties for CHMOD, CHOWN, TRUNCATE, UNLINK+re-create, and atomic rename-swap.

(3) Exhaustive concurrency testing (3 tests). Using `loom` 0.7, all thread interleavings are explored for: (a) two concurrent GTXNs with disjoint write sets both succeed; (b) a rolled-back GTXN restores pre-transaction state exactly; (c) conflicting writes to the same slot are serialized by the mutex.

(4) Crash consistency testing (7 tests). Using `redb` 2.1's ACID write transactions with `tempfile`-based databases: save/load round-trip preserves all nodes, edges, and invariants; sequential overwrites are atomic; data survives process restart (close + reopen); and complex trees with hard links and renames survive persist.

(5) Adversarial security scenarios (13 tests). Integration tests that simulate attacker behavior patterns:

- *Ransomware detection*: 50 files created then mass-renamed — the Ollivier–Ricci curvature monitor detects the structural anomaly (mean κ deviation exceeds threshold).

- *Hardlink bomb*: 100 hard links to a single inode — treewidth spikes above the baseline bound.
- *Deep nesting*: 200+ nested directories — treewidth remains bounded per Theorem 7.1.
- *Capability attenuation*: derived capabilities never escalate rights beyond their CDT ancestor.
- *Deceptive projections*: RESTRICT hides sibling subtrees; REDIRECT returns decoy data; FABRICATE adds readable honeypot files.

11.3 Benchmark Configuration

Three Criterion 0.5 benchmark suites measure operation latency at multiple scales, enabling regression detection and scaling analysis.

Table 5: Benchmark suite configuration (58 parametrized measurements).

Suite	Benchmarks	Scales
DPO operations	CREATE, MKDIR, UNLINK, RENAME, LINK, WRITE/READ, invariant check	10–1,000
WAL index	Linear scan vs. indexed lookup; DPO sequence replay	10–1,000
Monitors	Treewidth (star/chain/tree); curvature (full vs. incremental)	50–10,000

Expected complexity. Each DPO rule performs $O(1)$ arena operations plus $O(\log n)$ BTreeMap index updates, yielding $O(\log n)$ per operation. The invariant check is $O(n + m)$ where n and m are node and edge counts. Treewidth is $O(n^2)$ for a full recompute (greedy min-degree) but $O(\Delta^2)$ incremental. Ollivier–Ricci curvature is $O(|E| \cdot |V|)$ full or $O(\delta \cdot d_{\max})$ incremental, where δ is the number of affected edges and $d_{\max}$ is the maximum degree.

Methodology. All benchmarks use Criterion's default configuration: 100 samples per measurement, 5-second warm-up, outlier detection via modified Z-scores. Results are stored as JSON in `target/criterion/` for historical comparison across commits. The benchmark harness is invoked via `cargo bench -workspace -exclude sotfs-fuse`.

12 Conclusion and Future Work

SOTFS demonstrates that treating filesystem metadata as an explicit typed graph, with operations formalized as algebraic DPO rewriting rules, unlocks structural properties—bounded treewidth, discrete curvature monitoring, and capability-gated projections—that are impossible in conventional implicit-graph designs.

The immediate next steps are: (Phase 2) FUSE prototype on Linux implementing the type graph and DPO engine as Rust crates; (Phase 3) integration with SOTX as a native filesystem server; (Phase 4) incremental formal verification; (Phase 5) structural differentiators (treewidth enforcement, curvature monitoring, deceptive projections).

References

[1] H. Ehrig, K. Ehrig, U. Prange, G. Taentzer. *Fundamentals of Algebraic Graph Transformation*. Springer, 2006.

[2] B. Courcelle. "The Monadic Second-Order Logic of Graphs I: Recognizable Sets of Finite Graphs." *Information and Computation*, 85(1):12–75, 1990.

[3] Y. Ollivier. "Ricci Curvature of Markov Chains on Metric Spaces." *Journal of Functional Analysis*, 256(3):810–864, 2009.

[4] Y. Lin, L. Lu, S.-T. Yau. "Ricci Curvature of Graphs." *Tohoku Mathematical Journal*, 63(4):605–627, 2011.

[5] H. Chen, D. Ziegler, T. Chajed, A. Chlipala, M. F. Kaashoek, N. Zeldovich. "Using Crash Hoare Logic for Certifying the FSCQ File System." *SOSP*, 2015.

[6] H. Sigurbjarnarson, J. Bornholt, E. Torlak, X. Wang. "Push-Button Verification of File Systems via Crash Refinement." *OSDI*, 2016.

[7] S. Kadekodi, A. Ramanathan, V. Narayanan. "SquirrelFS: Using the Rust Compiler to Check File-System Crash Consistency." *OSDI*, 2024.

[8] R. N. M. Watson, J. Anderson, B. Laurie, K. Kennaway. "Capsicum: Practical Capabilities for UNIX." *USENIX Security*, 2010.

[9] G. Klein, K. Elphinstone, G. Heiser, et al. "seL4: Formal Verification of an OS Kernel." *SOSP*, 2009.

[10] T. Chajed, J. Tassarotti, M. Theng, R. Jung, M. F. Kaashoek, N. Zeldovich. "Verifying Concurrent, Crash-Safe Systems with Perennial." *SOSP*, 2019.

[11] N. Robertson, P. D. Seymour. "Graph Minors. II. Algorithmic Aspects of Tree-Width." *Journal of Algorithms*, 7(3):309–322, 1986.

[12] H. L. Bodlaender. "A Partial k-Arboretum of Graphs with Bounded Treewidth." *Theoretical Computer Science*, 209(1–2):1–45, 1998.